When we call animal.upcase, Ruby checks the object referenced by the animal variable and then looks up the method upcase in the class object referenced from the animal object. Our animal is a string and so all the methods of class String are available.

Now let's make it more interesting. We'll define a singleton method on the string referenced by animal. We've seen this syntax before, we use a method name that includes a reference to a specific object:

```
animal = "cat"
def animal.speak
  puts "The #{self} says miaow"
end

animal.speak
puts animal.upcase
```

produces:

```
The cat says miaow
CAT
```

The call to animal.speak is handled the same way as when we looked at how animal.upcase is invoked. Ruby sets self to the string object "cat" that is referenced by animal and then looks for a method called speak in that object's class. Surprisingly, it finds it. This is initially surprising because the class of "cat" is String, and String doesn't have a speak method. The specific object called animal, however, does have a speak method that we've defined for that object and that object only.

So, does Ruby have some kind of special-case magic for these methods that are defined on individual objects?

Not exactly.

When we defined animal.speak, the singleton method for the "cat" object, Ruby created a new anonymous class and placed the speak method in that class. This anonymous class goes by a couple of different names, but you'll most likely see it called a *singleton class* (it's sometimes called an *eigenclass*). We prefer the former name, because it ties in to the idea of singleton methods.

Every object in Ruby has the potential of having its own singleton class. When you define a singleton class, Ruby creates that anonymous class and makes it the singleton class of that object. You can access that singleton class via the singleton_class method, and you can get a list of methods defined there with singleton_methods.

```
animal = "cat"
def animal.speak
  puts "The #{self} says miaow"
end

animal.speak
puts animal.class
puts animal.singleton_class
puts animal.singleton_methods
```

produces:

```
The cat says miaow
String
#<Class:#<String:0x00000001047a8e60>>
```